디지털 설계 퀴즈 예상문제 2세트 (11/10) — 정답 포함

2026년 11월 8일 작성 · 노트의 퀴즈 범위 기준 · 1세트와 다른 문제 35개 · 정답 포함 버전

안내

1세트와 같은 범위에서 겹치지 않는 문제로 35개를 새로 뽑았어요. 1세트보다 코드 해석과 "왜 그런가"를 묻는 문제를 조금 더 넣었어요.

표시	유형	준비 포인트
[개념]	서술형	이유까지 2~3문장으로 설명할 수 있어야 해요
[해석]	코드 읽기	합성 결과나 시뮬레이션 값을 예측해요
[설계]	코드 작성	손으로 Verilog를 써 봐요

이 버전은 문제마다 바로 아래에 정답과 해설을 붙였어요. 복습할 때 보는 용도예요.

1. 조합 논리: MUX, Decoder

1-1. [설계] 8비트 4:1 MUX 모듈 `mux4`를 만들고, 이것 2개와 2:1 MUX 1개를 써서 8:1 MUX(`sel[2:0]`)를 **계층 구조(인스턴스)**로 작성하세요.

> **정답·해설**
>
> ```
> module mux4 (
>   input  [7:0] i0, i1, i2, i3,
>   input  [1:0] s,
>   output [7:0] y
> );
>   assign y = (s == 2'd0) ? i0 :
>              (s == 2'd1) ? i1 :
>              (s == 2'd2) ? i2 : i3;
> endmodule
>
> module mux8 (
>   input  [7:0] d0, d1, d2, d3, d4, d5, d6, d7,
>   input  [2:0] sel,
>   output [7:0] y
> );
>   wire [7:0] lo, hi;
>   mux4 u_lo (.i0(d0), .i1(d1), .i2(d2), .i3(d3), .s(sel[1:0]), .y(lo));
>   mux4 u_hi (.i0(d4), .i1(d5), .i2(d6), .i3(d7), .s(sel[1:0]), .y(hi));
>   assign y = sel[2] ? hi : lo;   // 2:1 MUX
> endmodule
> ```
>
> 아래 두 비트로 각 4:1 MUX에서 하나씩 고르고, 최상위 비트 `sel[2]`로 둘 중 하나를 골라요.

1-2. [설계] 출력이 Active-low인 3:8 디코더를 작성하세요. 입력 `a[2:0]`에 해당하는 출력 비트만 0이고 나머지는 1입니다.

> **정답·해설**

```
always @(*) begin
  y_n    = 8'hFF;   // 기본값: 모두 1 (Latch 방지)
  y_n[a] = 1'b0;    // 선택된 비트만 0
end
```

`case (a)`로 8가지를 모두 써도 정답이에요.

1-3. [해석] 아래 회로의 이름은 무엇입니까? `in = 4'b0110`일 때와 `in = 4'b0000`일 때 `out`, `valid` 값은?

```
always @(*) begin
  valid = 1'b1;
  if      (in[3]) out = 2'd3;
  else if (in[2]) out = 2'd2;
  else if (in[1]) out = 2'd1;
  else if (in[0]) out = 2'd0;
  else begin out = 2'd0; valid = 1'b0; end
end
```

정답·해설

4:2 우선순위 인코더(Priority encoder)예요. 위 비트일수록 우선순위가 높아요.

- `in = 4'b0110` : 켜진 비트 중 가장 높은 것이 `in[2]` → `out = 2`, `valid = 1`
- `in = 4'b0000` : 켜진 비트가 없음 → `out = 0`, `valid = 0`

`valid`가 없으면 "입력 없음"과 "in[0]만 1"을 구분할 수 없어요.

1-4. [개념] 같은 기능을 `if – else if` 체인으로 쓸 때와 `case`로 쓸 때 합성되는 회로 구조가 어떻게 다른지 설명하세요.

정답·해설

`if – else if`는 앞 조건이 먼저 검사되므로 **우선순위가 있는 MUX 체인**으로 합성돼요. 조건이 많을수록 MUX가 직렬로 길어져 지연이 늘어요. `case`는 조건이 서로 겹치지 않으면 **병렬 MUX** 하나로 합성되어 지연이 일정해요. 우선순위가 필요 없으면 `case`가 유리해요.

1-5. [설계] 4:1 MUX 하나로 F(A, B, C) = Σm(1, 2, 6, 7)을 구현하세요. A, B를 select로 쓰고 데이터 입력 I0~I3에는 C, ~C, 0, 1 중 하나를 넣습니다.

정답·해설

A, B로 4개 구간을 나누고 각 구간에서 F를 C로 표현해요.

A B	해당 minterm	F	입력
0 0	m0(0), m1(1)	C=1일 때만 1	I0 = C
0 1	m2(1), m3(0)	C=0일 때만 1	I1 = ~C
1 0	m4(0), m5(0)	항상 0	I2 = 0
1 1	m6(1), m7(1)	항상 1	I3 = 1

```
assign F = ({A, B} == 2'b00) ? C  :
           ({A, B} == 2'b01) ? ~C :
           ({A, B} == 2'b10) ? 1'b0 : 1'b1;
```

2. 순차 논리: Counter, tick_gen

2-1. [설계] 4비트 Up/Down 카운터를 작성하세요. 비동기 리셋 `rst`, 병렬 로드 `load`(입력 `din[3:0]`), 방향 `up`(1이면 증가, 0이면 감소)이 있고 `load`가 카운트보다 우선입니다.

정답·해설

```
always @(posedge clk or posedge rst) begin
  if (rst)       cnt <= 4'd0;
  else if (load) cnt <= din;        // 로드가 우선
  else if (up)   cnt <= cnt + 4'd1; // 15 → 0 자동 순환
  else           cnt <= cnt - 4'd1; // 0 → 15 자동 순환
end
```

2-2. [설계] 50 MHz 클럭으로 1초마다 `tick`을 만들고, 그 `tick`으로 0~59를 세는 초 카운터 `sec`를 작성하세요. 분주 카운터의 최대값과 비트 수도 구하세요.

정답·해설

50 MHz × 1초 = 50,000,000클럭이므로 분주 카운터는 0~49,999,999를 세고 **최대값은 49,999,999**예요. 2^25 = 33,554,432 < 50,000,000 ≤ 2^26 = 67,108,864이므로 **26비트**가 필요해요.

```
reg [25:0] div;
reg        tick;
reg [5:0]  sec;

always @(posedge clk or posedge rst) begin
  if (rst) begin
    div <= 0;  tick <= 1'b0;
  end else if (div == 26'd49_999_999) begin
    div <= 0;  tick <= 1'b1;
  end else begin
    div <= div + 1;  tick <= 1'b0;
  end
end

always @(posedge clk or posedge rst) begin
  if (rst)       sec <= 6'd0;
  else if (tick) sec <= (sec == 6'd59) ? 6'd0 : sec + 6'd1;
end
```

초 카운터도 같은 `clk`를 쓰고 `tick`은 enable로만 써요(1세트 2-4 참고).

2-3. [해석] 아래 카운터에 리셋 직후 `q = 4'b0000`입니다. 이어지는 상태를 차례로 쓰고, 이 카운터의 이름과 상태 수를 쓰세요.

```
always @(posedge clk)
  q <= {q[2:0], ~q[3]};
```

정답·해설

`0000 → 0001 → 0011 → 0111 → 1111 → 1110 → 1100 → 1000 → 0000` 순으로 돌아요. **Johnson 카운터(Twisted ring counter)**이고 상태는 **8개**예요. n비트 Johnson 카운터는 2n개 상태를 가져요.

2-4. [개념] 동기 카운터와 비동기(Ripple) 카운터의 차이를 설명하고, FPGA·ASIC 설계에서 Ripple 카운터를 피하는 이유를 쓰세요.

> **정답·해설**
>
> 동기 카운터는 모든 플립플롭이 **같은 클럭**으로 동시에 바뀌어요. Ripple 카운터는 앞 플립플롭의 출력이 다음 플립플롭의 클럭이 되어 지연이 **비트마다 누적**돼요. 그래서 중간에 잘못된 값(글리치)이 보이고, 비트마다 클럭이 달라 사실상 여러 클럭 도메인이 생겨 타이밍 분석이 어려워요. FPGA에서는 전용 클럭 배선도 쓰지 못해요.

3. F/F, Synchronizer, SIPO/PISO

3-1. [해석] 아래 코드는 어떤 하드웨어로 합성됩니까? `en = 0`일 때 값을 유지하는데 Latch가 아닌 이유는?

```
always @(posedge clk)
  if (en) q <= d;
```

> **정답·해설**
>
> **Enable이 있는 D 플립플롭**으로 합성돼요. 플립플롭의 Clock enable 핀을 쓰거나, `en`으로 `d`와 `q`(자기 출력)를 고르는 2:1 MUX가 D 입력 앞에 붙어요. 값 유지는 **클럭 엣지에서만** 일어나는 플립플롭 동작이므로 Latch가 아니에요. Latch는 조합 블록(`@(*)`)에서 값을 주지 않는 경로가 있을 때 생겨요.

3-2. [설계] 외부 버튼 입력 `btn`(비동기)을 받아, 0→1로 바뀔 때 1클럭 폭의 펄스 `rise`를 내는 회로를 작성하세요. 동기화기와 엣지 검출을 함께 넣습니다.

> **정답·해설**
>
> ```
> reg s1, s2, s3;
> always @(posedge clk or posedge rst) begin
>   if (rst) {s3, s2, s1} <= 3'b000;
>   else     {s3, s2, s1} <= {s2, s1, btn};
> end
> assign rise = s2 & ~s3;   // 현재 1, 한 클럭 전 0 → 상승 엣지
> ```
>
> `s1`, `s2`가 2-FF 동기화기이고 `s3`은 한 클럭 전 값이에요. 실제 버튼은 바운싱이 있어서 디바운싱 회로도 필요하다고 덧붙이면 좋아요.

3-3. [개념] 메타스테이빌리티(Metastability)가 무엇인지 설명하고, Setup time과 Hold time을 정의하세요.

> **정답·해설**
>
> 입력이 Setup·Hold 구간 안에서 바뀌면 플립플롭 출력이 0도 1도 아닌 중간 상태에 머물다가 **정해지지 않은 시간 뒤에** 어느 한쪽으로 정착하는 현상이에요.
>
> - **Setup time**: 클럭 엣지 **전에** 데이터가 안정되어 있어야 하는 최소 시간
> - **Hold time**: 클럭 엣지 **후에** 데이터가 유지되어야 하는 최소 시간

3-4. [설계] UART 수신처럼 **LSB부터** 들어오는 직렬 데이터를 받는 8비트 SIPO를 작성하세요. MSB부터 받는 SIPO와 코드가 어떻게 다른지도 쓰세요.

> **정답·해설**
>
> ```
> always @(posedge clk)
>   sr <= {din, sr[7:1]};   // 오른쪽으로 시프트: 새 비트가 MSB로 들어옴
> ```

8클럭 뒤 처음 들어온 비트(LSB)는 `sr[0]`에, 마지막 비트는 `sr[7]`에 놓여요. MSB부터 받을 때는 `sr <= {sr[6:0], din};` 처럼 반대 방향으로 시프트해요.

3-5. [해석] PISO에 `8'hC4`를 로드했습니다. (1) MSB부터 내보낼 때, (2) LSB부터 내보낼 때 8클럭 동안 나오는 비트 순서를 쓰세요.

정답·해설

`8'hC4` = `1100_0100`이에요.

1. MSB부터: **1, 1, 0, 0, 0, 1, 0, 0**
2. LSB부터: **0, 0, 1, 0, 0, 0, 1, 1**

3-6. [해석] 클럭 100 MHz, 플립플롭 Clock-to-Q 1 ns, 두 플립플롭 사이 조합 로직 지연 6 ns, Setup time 0.5 ns입니다. Setup slack은 얼마이고, 이 경로의 최대 동작 주파수는 약 얼마입니까?

정답·해설

클럭 주기는 10 ns예요. 필요한 시간은 1 + 6 + 0.5 = 7.5 ns이므로 **Setup slack = 10 − 7.5 = +2.5 ns**예요(양수라 통과). 최소 주기 7.5 ns → **최대 주파수 ≈ 133 MHz**예요.

4. FSM, Pattern Detector

4-1. [설계] 패턴 `110`을 검출하는 **Mealy** FSM을 설계하세요(overlap 허용). 상태 전이표를 쓰고 Verilog로 작성하세요.

정답·해설

상태는 S0(없음), S1(`1`), S2(`11`)예요. 출력은 전이에 붙어요(Mealy).

현재 상태	din = 0	din = 1
S0	S0 / 0	S1 / 0
S1 (`1`)	S0 / 0	S2 / 0
S2 (`11`)	**S0 / 1**	S2 / 0

S2에서 1이 또 오면 끝 두 비트가 여전히 `11`이라 S2에 머물러요.

```
localparam S0 = 2'd0, S1 = 2'd1, S2 = 2'd2;
reg [1:0] state, next;

always @(posedge clk or posedge rst)
  if (rst) state <= S0;
  else     state <= next;

always @(*) begin
  next   = S0;
  detect = 1'b0;           // 기본값 (Latch 방지)
  case (state)
    S0: next = din ? S1 : S0;
    S1: next = din ? S2 : S0;
    S2: if (din) next = S2;
        else begin next = S0; detect = 1'b1; end
    default: next = S0;
  endcase
end
```

4-2. [해석] `101` 검출기에 `1 0 1 0 1`을 넣었습니다. (1) overlap 허용, (2) overlap 비허용일 때 각각 몇 번 검출됩니까?

정답·해설

(1) overlap 허용: 3번째와 5번째 비트에서 **2번** 검출돼요. 3번째에서 검출한 `101`의 끝 `1`이 다음 패턴의 시작이 돼요. (2) overlap 비허용: 검출 뒤 S0으로 돌아가므로 남은 `0 1`로는 패턴이 안 되어 **1번**이에요.

4-3. [설계] 1초 `tick`을 입력으로 받는 신호등 Moore FSM을 설계하세요. 초록 5초 → 노랑 2초 → 빨강 4초를 반복하고, 출력은 `{red, yellow, green}` 입니다.

정답·해설

```
localparam G = 2'd0, Y = 2'd1, R = 2'd2;
reg [1:0] state, next;
reg [2:0] cnt;    // 현재 상태에서 지난 초

always @(posedge clk or posedge rst) begin
  if (rst) begin
    state <= G;  cnt <= 3'd0;
  end else if (tick) begin
    cnt   <= (next != state) ? 3'd0 : cnt + 3'd1;
    state <= next;
  end
end

always @(*) begin
  next = state;
  case (state)
    G: if (cnt == 3'd4) next = Y;   // 5초
    Y: if (cnt == 3'd1) next = R;   // 2초
    R: if (cnt == 3'd3) next = G;   // 4초
    default: next = G;
  endcase
end

assign {red, yellow, green} = (state == G) ? 3'b001 :
                              (state == Y) ? 3'b010 : 3'b100;
```

출력이 상태에만 의존하므로 Moore예요. 카운트 비교값이 "시간 − 1"인 이유는 0부터 세기 때문이에요.

4-4. [개념] FSM 출력에 글리치가 생길 수 있는 경우를 쓰고, 이를 막는 방법(Registered output 등)을 설명하세요.

정답·해설

Mealy 출력은 입력이 바뀌면 바로 변해서 입력 글리치가 출력으로 전달돼요. Moore라도 여러 상태 비트를 조합 로직으로 디코딩하면, 상태 비트들이 미세하게 다른 시각에 바뀌는 순간 잘못된 값이 잠깐 나와요. 막는 방법은 다음과 같아요.

1. 출력을 플립플롭으로 한 번 더 받는 **Registered output**을 써요(1클럭 지연, 또는 다음 상태로 미리 계산해 지연 없이).
2. 출력 디코딩이 단순한 **One-hot 인코딩**이나 출력 비트를 상태 코드에 직접 넣는 인코딩을 써요.

5. 설계 가이드: Latch

5-1. [해석] 아래 코드에서 Latch가 생기는 신호와 그 상태를 모두 찾고 고치세요.

```
always @(*) begin
  case (state)
    IDLE: begin led = 1'b0; busy = 1'b0; end
    RUN:  begin led = 1'b1; end
    DONE: begin busy = 1'b1; end
    default: begin led = 1'b0; busy = 1'b0; end
  endcase
end
```

정답·해설

`led` 는 **DONE** 상태에서, `busy` 는 **RUN** 상태에서 값이 할당되지 않아 둘 다 Latch가 생겨요. `case` 앞에 기본값을 두면 고쳐져요.

```
always @(*) begin
  led  = 1'b0;   // 기본값
  busy = 1'b0;
  case (state)
    RUN:  led  = 1'b1;
    DONE: busy = 1'b1;
    default: ;
  endcase
end
```

5-2. [해석] `always @(*) if (en) q = d;` 는 어떤 소자가 됩니까? 이 코드를 `always_comb` 로 쓸 때와 `always_latch` 로 쓸 때 툴의 반응은 어떻게 다릅니까?

> **정답·해설**
> `en` 이 1일 때 `d` 가 그대로 통과하고 0이면 값을 유지하는 **D Latch**가 돼요. `always_comb` 로 쓰면 "조합 블록인데 Latch가 추론됐다"는 경고나 에러가 나요. `always_latch` 로 쓰면 Latch가 의도라는 뜻이라 툴이 정상으로 받아들여요.

5-3. [개념] 조합 루프(Combinational loop)란 무엇이고, `assign a = ~a;` 같은 코드가 문제가 되는 이유를 쓰세요.

> **정답·해설**
> 플립플롭 없이 조합 로직의 출력이 다시 자기 입력으로 돌아오는 구조예요. `a = ~a` 는 값이 계속 뒤집혀 **발진**하거나 값이 정해지지 않아요. 시뮬레이션은 0 지연 무한 루프로 멈출 수 있고, 타이밍 분석(STA)도 할 수 없어요. 루프 안에 플립플롭을 넣어 끊어야 해요.

5-4. [개념] 클럭을 끄려고 `assign gclk = clk & en;` 처럼 직접 게이팅하면 안 되는 이유를 쓰고, 대신 쓸 수 있는 방법 두 가지를 쓰세요.

> **정답·해설**
> `clk` 가 1인 동안 `en` 이 바뀌면 `gclk` 에 **짧은 펄스(글리치)**가 생겨 플립플롭이 잘못 동작해요. AND 게이트 지연 때문에 다른 플립플롭과 **클럭 skew**도 생겨요. 대신 쓸 방법은 다음과 같아요.
>
> 1. 클럭은 그대로 두고 플립플롭의 **Clock enable**(`if (en) q <= d;`)을 써요.
> 2. ASIC은 Latch 기반 **ICG(Integrated Clock Gating) 셀**, FPGA는 BUFGCE 같은 전용 클럭 버퍼를 써요.

5-5. [개념] O/X 설계 가이드에 맞으면 O, 틀리면 X를 하세요.

1. `initial` 블록의 초기값은 ASIC 합성에서도 항상 반영된다.
2. `#10` 같은 지연 구문은 합성할 때 무시된다.
3. 하나의 `always` 블록에는 하나의 클럭 엣지만 쓰는 것이 좋다.
4. 조합 논리의 모든 경로에서 모든 출력에 값을 주면 Latch가 생기지 않는다.
5. 리셋이 없는 레지스터는 절대 쓰면 안 된다.
6. `for` 문은 하드웨어에서 클럭마다 반복 실행되는 루프가 된다.

> **정답·해설**
> 1 **X** (ASIC 합성은 `initial` 을 무시해요. 일부 FPGA만 초기값을 지원해서 리셋으로 초기화하는 게 원칙이에요) · 2 **O** · 3 **O** · 4 **O** · 5 **X** (데이터 경로 레지스터는 리셋 없이 쓰기도 해요. 제어 신호와 FSM에는 리셋이 필요해요) · 6 **X** (`for` 는 펼쳐져서(unroll) 하드웨어가 복제돼요)

6. Blocking/Non-blocking, Race condition

6-1. [해석] `a = 1`, `b = 0`, `c = 0`에서 클럭 엣지가 한 번 올라왔습니다. 엣지 뒤 `b`, `c` 값은? 이런 코드를 피해야 하는 이유는?

```
always @(posedge clk) begin
  b <= a;
  c = b;
end
```

> **정답·해설**
>
> **`b = 1, c = 0`**이에요. `b <= a`는 블록이 끝난 뒤에 반영되므로, 그 전에 실행되는 `c = b`는 옛 `b`(0)를 읽어요. 한 블록에서 `=`와 `<=`를 섞으면 결과를 예측하기 어렵고, 다른 블록에서 `c`를 읽으면 Race도 생길 수 있어요. 순차 블록은 모두 `<=`로 써요.

6-2. [해석] 조합 논리를 아래처럼 Non-blocking으로 썼습니다. 어떤 문제가 생길 수 있고, 가이드는 무엇입니까?

```
always @(*) begin
  t <= a & b;
  y <= t | c;
end
```

> **정답·해설**
>
> `t`가 NBA 영역에서 늦게 바뀌어서 `y`는 처음에 옛 `t`로 계산돼요. `@(*)`에 `t`가 들어 있어 블록이 다시 실행되면 결국 맞는 값이 되지만, 불필요한 재실행이 생겨요. sensitivity list를 `@(a or b or c)` 처럼 직접 쓰면 `y`가 틀린 값으로 남아 합성 결과와 달라져요. 가이드는 **조합 논리는 Blocking(`=`)**이에요.

6-3. [해석] 아래 코드는 시프트 레지스터로 동작합니까? 동작한다면 그래도 권장하지 않는 이유는?

```
always @(posedge clk) begin
  q3 = q2;
  q2 = q1;
  q1 = d;
end
```

> **정답·해설**
>
> 뒤 단부터 먼저 대입하므로 각 문장이 아직 바뀌지 않은 앞 단 값을 읽어서 **시프트 레지스터로 동작해요**. 하지만 문장 순서를 바꾸면 바로 깨지고, 다른 `always` 블록에서 `q1~q3`를 읽으면 Race가 생겨요. 순서와 상관없이 맞게 동작하는 `<=`를 써야 해요.

6-4. [개념] `q <= d;`로 바뀌는 값을 같은 시각에 `$display`와 `$strobe`로 출력하면 결과가 어떻게 다릅니까? 이벤트 큐(Active, NBA 영역)를 들어 설명하세요.

> **정답·해설**
>
> `$display`는 실행되는 순간(**Active 영역**) 출력해서 아직 NBA가 반영되기 전인 **옛 값**을 보여 줘요. `$strobe`는 그 시각의 모든 처리(NBA 포함)가 끝난 뒤 출력해서 **새 값**을 보여 줘요. `$monitor`도 시각이 끝날 때 값이 바뀐 경우에만 출력해요.

6-5. [해석] 아래 테스트벤치에서 t = 10에 `q`에 들어가는 값은 무엇입니까? 문제가 있다면 고치세요.

```
initial clk = 1;
always #5 clk = ~clk;          // t = 10, 20, ...에 상승 엣지
initial begin a = 0; #10 a = 1; end
always @(posedge clk) q <= a;   // DUT라고 가정
```

> **정답·해설**
>
> t = 10에 `a = 1`과 `posedge clk`이 같은 시각에 일어나서 **Race**예요. 시뮬레이터가 어느 쪽을 먼저 처리하느냐에 따라 `q`가 0일 수도 1일 수도 있어요. 고치는 방법은 다음과 같아요.
>
> 1. `a`를 클럭 엣지와 다른 시각에 바꿔요(예: `#12 a = 1;` 또는 `@(negedge clk)`).
> 2. 테스트벤치에서도 `a <= 1;`로 바꾸면 DUT가 엣지에서 옛 값(0)을 확정적으로 읽어요.

7. Reset, CDC

7-1. [해석] Active-low 리셋 `rst_n`을 쓰는 아래 코드의 버그를 찾고 고치세요.

```
always @(posedge clk or negedge rst_n)
  if (rst_n) q <= 1'b0;
  else       q <= d;
```

> **정답·해설**
>
> 조건이 반대예요. Active-low 리셋이므로 `rst_n`이 **0일 때** 리셋해야 해요. 지금 코드는 정상 동작(`rst_n = 1`) 중에 클럭마다 `q`를 0으로 지워 버려요.
>
> ```
> always @(posedge clk or negedge rst_n)
>   if (!rst_n) q <= 1'b0;
>   else        q <= d;
> ```

7-2. [개념] 비동기 리셋 입력에 짧은 글리치가 들어오면 어떤 일이 생깁니까? 대책은?

> **정답·해설**
>
> 비동기 리셋은 클럭과 상관없이 바로 반응해서, 짧은 글리치에도 일부 또는 모든 플립플롭이 **동작 중에 리셋**돼요. 일부만 리셋되면 시스템 상태가 어긋나요. 외부 리셋 입력에 글리치 필터(디바운스)를 두고, 리셋 동기화기와 깨끗한 리셋 배선으로 내부에 분배해요.

7-3. [설계] `clk_a` 도메인의 1클럭 펄스 `pulse_a`를 `clk_b` 도메인의 1클럭 펄스 `pulse_b`로 넘기는 Toggle synchronizer를 작성하세요. 이 방식을 쓸 수 있는 조건도 쓰세요.

> **정답·해설**

```
// clk_a 도메인: 펄스마다 레벨을 뒤집음
reg tog;
always @(posedge clk_a or negedge rst_n)
  if (!rst_n)       tog <= 1'b0;
  else if (pulse_a) tog <= ~tog;

// clk_b 도메인: 2-FF 동기화 + 엣지 검출
reg s1, s2, s3;
always @(posedge clk_b or negedge rst_n)
  if (!rst_n) {s3, s2, s1} <= 3'b000;
  else        {s3, s2, s1} <= {s2, s1, tog};

assign pulse_b = s2 ^ s3;   // 레벨이 바뀐 순간 1클럭 펄스
```

조건: 펄스와 펄스 사이 간격이 받는 쪽에서 동기화하고 검출할 만큼(대략 `clk_b` 2~3주기 이상) 떨어져 있어야 해요. 너무 빨리 연속으로 오면 토글 두 번이 합쳐져 펄스를 놓쳐요.

7-4. [개념] 2-FF Synchronizer 바로 앞에 조합 로직을 두면 안 되는 이유는?

정답·해설

조합 로직 출력은 입력이 바뀌는 동안 **글리치**가 생길 수 있어요. 동기화기는 그 글리치를 정상 신호로 잡아 버릴 수 있어요. 그래서 보내는 쪽에서 반드시 **플립플롭 출력(Registered)**으로 만든 신호를 동기화기에 넣어야 해요.

7-5. [개념] 느린 클럭 → 빠른 클럭으로 넘길 때와, 빠른 클럭 → 느린 클럭으로 넘길 때 단일 비트 신호의 위험이 어떻게 다른지 설명하세요.

정답·해설

느린 → 빠른: 신호가 최소 느린 클럭 한 주기 동안 유지되고, 그 사이 빠른 클럭 엣지가 여러 번 오므로 2-FF 동기화기로 안정적으로 잡혀요. 빠른 → 느린: 빠른 도메인의 짧은 펄스가 느린 클럭 엣지 사이에서 끝나 버려 **놓칠 수 있어요**. 펄스 늘리기, Toggle synchronizer, Handshake가 필요해요.

7-6. [해석] (1) Binary `0110`을 Gray code로, (2) Gray code `1101`을 Binary로 바꾸세요. Binary → Gray 변환을 Verilog 한 줄로 쓰세요.

정답·해설

1. Binary `0110` → Gray: `0110 ^ 0011` = **`0101`**
2. Gray `1101` → Binary: b3 = 1, b2 = 1^1 = 0, b1 = 0^0 = 0, b0 = 0^1 = 1 → **`1001`** (검산: `1001 ^ 0100` = `1101`)

```
assign gray = bin ^ (bin >> 1);
```